

[Secure Exit](#)SIMATS
ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING

CHITTIBOINA HARSHITHA
192473031RA

CO3-AT3-Debugging and Optimization

[View Only](#)[← Back](#)

QUESTIONS

Q1ATM Withdrawal – Exception
Handling Debugging
10 marks**Q2**Hospital Registration –
NullPointerException
Debugging
10 marks**Q3**E-Commerce Product Search –
Array Exception Debugging
10 marks**Q4**Bank Account – Race Condition
Debugging
10 marks**Q5**Railway Reservation –
Synchronization Debugging
10 marks

5/5 answered

Q5

Railway Reservation – Synchronization Debugging

10
marks

A railway reservation system has only one available seat. Two customers attempt to reserve the seat simultaneously.

The following complete Java program contains a synchronization problem.

Task:

1. Identify the race condition.
2. Debug the program using the synchronized keyword.
3. Ensure that only ONE customer can reserve the available seat.
4. The second customer must receive "No seat available".
5. Submit the COMPLETE corrected Java program.

Faulty Program:

```
class Reservation {  
  
    int seats = 1;  
  
    void bookSeat(String customer) {  
  
        if (seats > 0) {
```

```
System.out.println(customer
+ " is booking a seat");

try {
    Thread.sleep(100);
}
catch (InterruptedException e) {
    System.out.println("Thread interrupted");
}

seats--;

System.out.println(customer
+ " booking successful");

} else {
    System.out.println(customer
+ " - No seat available");
}
}
}

class Customer extends Thread {

    Reservation reservation;
    String customerName;

    Customer(Reservation reservation, String
customerName) {
        this.reservation = reservation;
        this.customerName = customerName;
    }

    public void run() {
        reservation.bookSeat(customerName);
    }
}

class RailwayReservation {

    public static void main(String[] args) {

        Reservation reservation = new Reservation();
```

```

Customer c1 =
new Customer(reservation, "Customer 1");

Customer c2 =
new Customer(reservation, "Customer 2");

c1.start();
c2.start();
}
}

```

The corrected program must protect the shared seat resource using synchronization.

🔧 Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```

1 import java.util.Scanner;
2 class Reservation
3 {
4     int seats = 1;
5     synchronized void bookSeat(String custom
6     {
7         if(seats>0)
8         {
9             System.out.println(customer +"is
10            try
11            {
12                Thread.sleep(100);
13            }
14            catch (InterruptedException e)
15            {
16                System.out.println("invalid
17            }
18            seats--;
19            System.out.println(customer + "b
20        }
21        else
22        {
23            System.out.println(customer +"no
24

```

```
25     }
26 }
27 public static void main(String[] args)
28 {
29     Reservation reservation = new Reserv
30     customer c1 = new customer(reservati
31     customer c2 = new customer(reservati
32     c1.start();
33     c2.start();
34 }
35 }
36 class customer extends Thread
37 {
38     Reservation reservation;
39     String customerName;
40     customer(Reservation reservation, String cust
41 {
42     this.reservation = reservation;
43     this.customerName = customerName;
44 }
45 public void run()
46 {
47     reservation.bookSeat(customerName);
48 }
49 }
```

Test Input

stdin input (optional)

Output

```
customer1is booking a seat
customer1booking successful
customer2no seats available
```

 Pending manual review

✓ Submitted

© 2026 **SIMATS Online Lab**. All rights reserved.
